

C Pointers, Arrays, and Generic Memory

Merged sheet based on ECM2433 arrays/strings, functions with pointers, dynamic memory, 2D arrays, pointer arithmetic, and generic void-pointer programming material.

What You Need To Know

1. Pointers, dereferencing, and caller updates

A pointer stores an address. The address-of operator gets an address; dereferencing accesses the object at that address. C passes arguments by value, so a function updates the caller's variable by receiving a pointer.

```
void set_to_42(int *p) {
    *p = 42;
}

int value = 5;
set_to_42(&value);
```

2. Arrays, strings, and allocation

Arrays are contiguous. A C string is a character array ending in `'\0'`. Dynamic memory from `malloc` must be checked and later freed.

```
char s[] = "Hello";
printf("%zu\n", sizeof s); /* 6: includes '\0' */
printf("%zu\n", strlen(s)); /* 5: visible characters */

int *a = malloc(n * sizeof *a);
if (a == NULL) {
    return 1;
}
free(a);
```

3. 2D arrays and simulations

A true 2D array parameter needs the column count in the type. A dynamically allocated `int ** grid` uses one allocation for the row pointers and one allocation per row. Clean up earlier rows if a later allocation fails.

```
void print_grid(int rows, int cols, int grid[rows][cols]) {
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            printf("%d ", grid[r][c]);
        }
        printf("\n");
    }
}
```

4. Generic memory with void pointers

`void *` can point at data of any object type, but it has no element size. Generic algorithms usually receive an element size and a comparison function. Cast to `char *` for byte-wise pointer arithmetic.

```
void swap_bytes(void *a, void *b, size_t size) {
    char *pa = a;
    char *pb = b;
    for (size_t i = 0; i < size; i++) {
        char tmp = pa[i];
        pa[i] = pb[i];
        pb[i] = tmp;
    }
}
```

Practice Questions

- 1 In `int x = 4; int *p = &x;`, identify the stored integer, the address expression, the pointer value, and the dereferenced value.
- 2 Write a function that sets a caller-owned integer to 42 through a pointer parameter.
- 3 Write a pointer-based integer swap function.
- 4 For `char s[20] = "Hello";`, compare `sizeof s` and `strlen(s)`.
- 5 Write code that allocates `n` integers, checks the allocation, fills them with `0 . . n-1`, and frees them.

Question 6 - explain the stale pointer:

```
int *p = malloc(sizeof *p);
*p = 7;
```

```
free(p);  
return *p;
```

- 6 Explain why the final dereference is invalid and name the kind of behaviour C gives.

Question 7 - complete manual string duplication:

```
char *duplicate(const char *src) {  
    char *copy = malloc(/* size here */);  
    if (copy == NULL) {  
        return NULL;  
    }  
    /* copy the string */  
    return copy;  
}
```

- 7 Complete the allocation size and copy operation.
- 8 Write the function header for a function that receives a 2D automatic array with rows and cols known at runtime.
- 9 In a Game-of-Life style update, why should you write the next generation into a separate grid instead of changing the current grid in place?
- 10 Write the key pointer-arithmetic expression for the address of element `i` in a generic array with base pointer `base` and element size `size`.

Mark Scheme

Attempt all questions before checking.

- 1 `x` is the integer object. `&x` is its address. `p` stores that address. `*p` is the integer stored there.
- 2 `void set_to_42(int *p) { *p = 42; }`, called with `set_to_42(&value);`
- 3 `void swap(int *a, int *b) { int tmp = *a; *a = *b; *b = tmp; }`
- 4 `sizeof s` is 20 because it is the whole array. `strlen(s)` is 5 because it stops before the null terminator.
- 5 Expected pattern: `int *a = malloc(n * sizeof *a); if (a == NULL) return 1; for (...) a[i] = i; free(a);`
- 6 After `free(p)`, the allocation no longer belongs to the program. `p` is stale/dangling, and `*p` is undefined behaviour.
- 7 Use `malloc(strlen(src) + 1)`, check for `NULL`, then `strcpy(copy, src);`.
- 8 `void f(int rows, int cols, int grid[rows][cols])`
- 9 If you update in place, later cells may read already-changed neighbour values. A separate next grid keeps all decisions based on the original generation.
- 10 `(char *)base + i * size`. Cast to `char *` because char pointer arithmetic advances one byte at a time.